

Investigación de Conceptos

Formulario Reactivo

Los formularios reactivos son una forma de crear formularios en angular de una forma escalable, y este lo que hace es que en vez de depender solo de HTML ósea dejarlo hacer todo esto lo que hace es que la lógica este apartada en un archivo de TypeScript. Este ayuda para tener un mejor control en validaciones complejas, el manejo de datos, mayor escalabilidad, pruebas de forma más fácil, y formularios dinámicos

las características son:

Los formularios lo que usan en TypeScript y usan clases u objetos.

```
1 nombre = new FormControl('');
```

También hay validaciones que están integradas con angular y estos se usan en los formularios para poder interactuar como, por ejemplo:

```
1 Validators.required
2 Validators.minLength(3)
3 Validators.email
```

En estos casos el primero sirve para que los campos sean obligatorios, el segundo es para agregarle un mínimo de caracteres (en este caso 3), y el 3ero funciona para que tenga la estructura del @gmail.com.

También podemos controlar los formularios con

Valid: este lo que hace es validar,

Invalid: este lo que hace es inhabilitar,

Dirty: este puede modificar datos,

Touched: Este lo que hace es verificar si el usuario a realizado un click.

```
1 if (this.formulario.valid) {
2   console.log("Formulario correcto");
3 }
```

Aquí está validando los datos del formulario que cumplen las reglas de validación asignadas y si es correcto entonces muestra el mensaje formulario correcto.

Reactive form module(Marcos-cloude)

ReactiveFormsModule basicamente es la habilidad reactiva del formulario, junto a su capacidad de no basarse unicamente en el componente HTML, sino que tambien poder controlarlo desde el componente TypeScript usando diferentes clases.

Ejemplo:

Componente TypeScript:

```
typescript

import { Component } from '@angular/core';
import { FormGroup, FormControl, Validators, ReactiveFormsModule } from '@angular/forms';

@Component({
  selector: 'app-registro',
  standalone: true,
  imports: [ReactiveFormsModule],
  templateUrl: './registro.component.html'
})
export class RegistroComponent {
  registroForm = new FormGroup({
    nombre: new FormControl('', [Validators.required, Validators.minLength(3)]),
    correo: new FormControl('', [Validators.required, Validators.email]),
  });

  onSubmit() {
    if (this.registroForm.valid) {
      console.log('Datos enviados:', this.registroForm.value);
    }
  }
}
```

Template HTML:

```
html

<form [formGroup]="registroForm" (ngSubmit)="onSubmit()">
  <input formControlName="nombre" placeholder="Nombre">
  <input formControlName="correo" placeholder="Correo">

  <button type="submit" [disabled]="registroForm.invalid">Enviar</button>
</form>
```

Dentro de sus aplicaciones tenemos cosas que requieren cierta complejidad en su código, cosas como un login, un register, etc.

Form Group

FormGroup es una clase de Angular (del módulo @angular/forms) que agrupa varios controles de formulario (FormControl) en una sola unidad. Representa un formulario o una sección de un formulario como un objeto cuyas propiedades son los controles individuales.

Qué hace

- Agrupa controles relacionados.
- Mantiene el valor combinado de todos los controles en un único objeto.
- Calcula y expone el estado de validez del grupo completo (válido solo si todos los controles hijos son válidos).
- Permite aplicar validadores a nivel de grupo.
- Facilita la sincronización de valores, el reseteo y la detección de cambios del conjunto de controles.

Ejemplo básico

```
import { FormGroup, FormControl } from '@angular/forms';

const formulario = new FormGroup({
  nombre: new FormControl(''),
  email: new FormControl(''),
  edad: new FormControl(0)
});
```

Ejemplo con validadores

```
import { FormGroup, FormControl, Validators } from '@angular/forms';

const formulario = new FormGroup({
  nombre: new FormControl('', Validators.required),
  email: new FormControl('', [Validators.required, Validators.email]),
  edad: new FormControl(null, [Validators.required, Validators.min(18)])
});
```

Ejemplo en un componente

```
import { Component } from '@angular/core';
import { FormGroup, FormControl, Validators } from '@angular/forms';

@Component({
  selector: 'app-registro',
  template: `
    <form [formGroup]="formulario" (ngSubmit)="enviar()">
      <input formControlName="nombre" placeholder="Nombre">
      <input formControlName="email" placeholder="Email">
      <button type="submit" [disabled]="formulario.invalid">Enviar</button>
    </form>
  `
})
export class RegistroComponent {
  formulario = new FormGroup({
    nombre: new FormControl('', Validators.required),
    email: new FormControl('', [Validators.required, Validators.email])
  });

  enviar() {
    console.log(this.formulario.value);
  }
}
```

Form Control:

Se utiliza para el manejo de los controles de formularios individuales. Permitiendo registrar valores dentro de un campo, verificar si ha sido modificada, validar el contenido y determinando su validación o sus errores.

Propiedades claves:

Value: Devuelve el valor que este asignado actualmente.

Valid / invalid: Validaciones que se cumplen o no.

touched: Indica si el usuario interactuó con el campo asignado.

dirty: Indica si el valor original ha sido modificado por el cliente.

TypeScript

```
import { Component } from '@angular/core';
import { FormControl, Validators } from '@angular/forms';

@Component({
  selector: 'app-manzanas',
  templateUrl: './manzanas.component.html'
})
export class ManzanasComponent {
  controlManzanas = new FormControl(5, [Validators.min(1)]);

  obtenerManzanas() {
    return this.controlManzanas.value;
  }

  actualizarManzanas(nuevaCantidad: number) {
    this.controlManzanas.setValue(nuevaCantidad);
  }

  reiniciarControl() {
    this.controlManzanas.reset();
  }
}
```

HTML

```
<div>
  <label for="manzanasInput">Cantidad de manzanas:</label>
  <input id="manzanasInput" type="number" [formControl]="controlManzanas">

  <p>Manzanas actuales: {{ controlManzanas.value }}</p>

  <button (click)="actualizarManzanas(10)">Establecer 10 manzanas</button>
  <button (click)="reiniciarControl()">Vaciar caja</button>

  <p *ngIf="controlManzanas.invalid">Debes tener al menos una manzana.</p>
</div>
```

Form Builder

Significa literalmente "Constructor Formularios", que sirve para poder crear formularios visualmente, sin programar demasiado, puede ser un componente o clase definiendo campos, validaciones y valores, en Angular se usa prácticamente para la creación de formularios reactivos.

Form Control Name

Es una directiva de Angular que conecta un `<input>` con un `FormControl` que existe dentro de un `FormGroup` el cual debe ya de estar definido dentro de Angular, Por ejemplo, si tienes un formulario con nombre y correo, `formControlName` le dice a Angular qué campo de tu formulario corresponde a cada `<input>`.

```
<form [formGroup]="userForm" (ngSubmit)="guardar()">

  <label>Nombre:</label>
  <input type="text" formControlName="nombre">

  <label>Email:</label>
  <input type="email" formControlName="email">

  <label>Edad:</label>
  <input type="number" formControlName="edad">
  <button type="submit">Guardar</button>
</form>
```

```
import { Component } from '@angular/core';
import { FormBuilder, Validators } from '@angular/forms';

@Component({
  selector: 'app-user-form',
  templateUrl: './user-form.component.html'
})
export class UserFormComponent {

  userForm = this.fb.group({
    nombre: ['', Validators.required],
    email: ['', [Validators.required, Validators.email]],
    edad: [null, Validators.min(18)]
  });

  constructor(private fb: FormBuilder) {}

  guardar() {
    if (this.userForm.valid) {
      console.log(this.userForm.value);
    }
  }
}
```

```
import { Component } from '@angular/core';
import { FormBuilder, Validators } from '@angular/forms';

@Component({
  selector: 'app-user-form',
  templateUrl: './user-form.component.html'
})
export class UserFormComponent {

  // Creamos el formulario
  userForm = this.fb.group({
    nombre: ['', Validators.required],
    email: ['', [Validators.required, Validators.email]]
  });

  constructor(private fb: FormBuilder) {}

  guardar() {
    console.log(this.userForm.value);
  }
}
```

le se
viado.
aracion
este

puede predecir el comportamiento por defecto del navegador (que normalmente recarga la pagina) y logra integrarse más facilme entre los otros componentes de angular.

```
<form [formGroup]="userForm" (ngSubmit)="guardar()">

  <label>Nombre:</label>

  <input
    type="text"
    formControlName="nombre"
  >

  <br><br>

  <label>Email:</label>

  <input
    type="email"
    formControlName="email"
  >

  <br><br>

  <button type="submit">
    Guardar
  </button>

</form>
```

Ejemplo

html

```
<form [formGroup]="miFormulario" (ngSubmit)="guardarDatos()">
  <input formControlName="usuario" placeholder="Usuario">
  <button type="submit">Guardar</button>
</form>
```

typescript

```
guardarDatos() {
  if (this.miFormulario.valid) {
    console.log(this.miFormulario.value);
  } else {
    console.log('Formulario inválido');
  }
}
```

También es común pasar el propio formulario como argumento (más habitual en *template-driven forms*):

html

```
<form #miForm="ngForm" (ngSubmit)="enviar(miForm)">
  <input name="usuario" ngModel required>
  <button type="submit">Enviar</button>
</form>
```

Aplicación práctica

- **Formularios de contacto o soporte:** al enviar, se valida y se hace una petición HTTP (POST) al backend.
- **Formularios multi-paso:** ngSubmit dispara la validación final antes de avanzar o guardar todo el proceso.

Prompt utilizados por cada uno:

- Marcos-Claude: Hola, requiero investigar a fondo los siguientes conceptos, para eso requiero una explicación de la funcionalidad del concepto, junto a un ejemplo y una aplicación de dichoso concepto de angular: Reactive form module, NgSubmit.
- Angel-ChatGPT: Hola, necesito que me des toda la información acerca de los siguientes temas: FormBuilder y FormControlName, necesito ejemplos de su implementación de Angular y dentro del HTML.

- Juan-Gemini: Buen día mi estimado, soy un estudiante y necesito que me des orientación sobre que es un Form Control en angular y ejemplos, necesito solo el texto con algunos ejemplos de código que utilizando manzanas para poder explicarlo mejor, no utilices emojis ni comentes el código para que yo pueda entenderlo mejor. Por favor
- Alex-copilot: Hola copilot, soy un estudiante de 5to perito en informática, ayudame con una investigación sobre los formularios reactivos en angular y ejemplos, necesito solo un texto con ejemplo de código de la rana pepe, para poder explicar de mejor manera esto no uses emojis ya que pueden llegar a ser confusos y así sea más simple.
- Aquino-Grok: Hola grok, soy un estudiante de informática y estoy haciendo un trabajo en grupo sobre investigar temas de angular y a mí me tocó investigar Form Group, quiero que me des una definición de que es, que hace y que me des ejemplos simples en código, no quiero que me generes ningún documento ni que uses emojis o que comentes el código. Por favor

Opinion: usar la IA para trabajar acorta el tiempo de trabajo al tener que solo revisar y copiar la información o código, pero hay que ser minuciosos al revisar código, también está el hecho de que hay que armar un buen prompt para que la IA no se confunda y no termine arruinando el código, en mi opinión es mucho más cómodo y fácil trabajar de esta forma siempre y cuando se conozca que se está haciendo.